深圳河套學院
Shenzhen Loop Area Institute

Natural Language Processing and Large Language Models

Xiaojun Quan and Zhengjun Yue

Shenzhen Loop Area Institute

Autumn, 2026

Why Do We Need LLM Alignment?

User Question: *Who won the 2023 Nobel Prize in Mathematics*?

LLM Response: *The 2023 Nobel Prize in Mathematics was awarded to Terence Tao for his groundbreaking contributions to mathematical analysis.*

User Question: 孙悟空为什么三打林黛玉?

LLM Response: 因为林黛玉多次被妖怪附身。第一次，孙悟空发现她身上有妖气；第二次，林黛玉试图迷惑唐僧；第三次，孙悟空识破了妖怪的真身最终将其消灭。这一情节体现了孙悟空嫉恶如仇的性格.

Why Do We Need LLM Alignment?

- **Knowledge hallucination** (知识幻觉) occurs when an LLM generates plausible (可信的) but factually incorrect results, as if it were true.

  一本正经地胡说八道

- **Alignment**: LLMs should not only generate plausible responses, but also provide truthful answers and acknowledge uncertainty when appropriate.

LLM alignment makes AI more helpful, truthful, and safe.

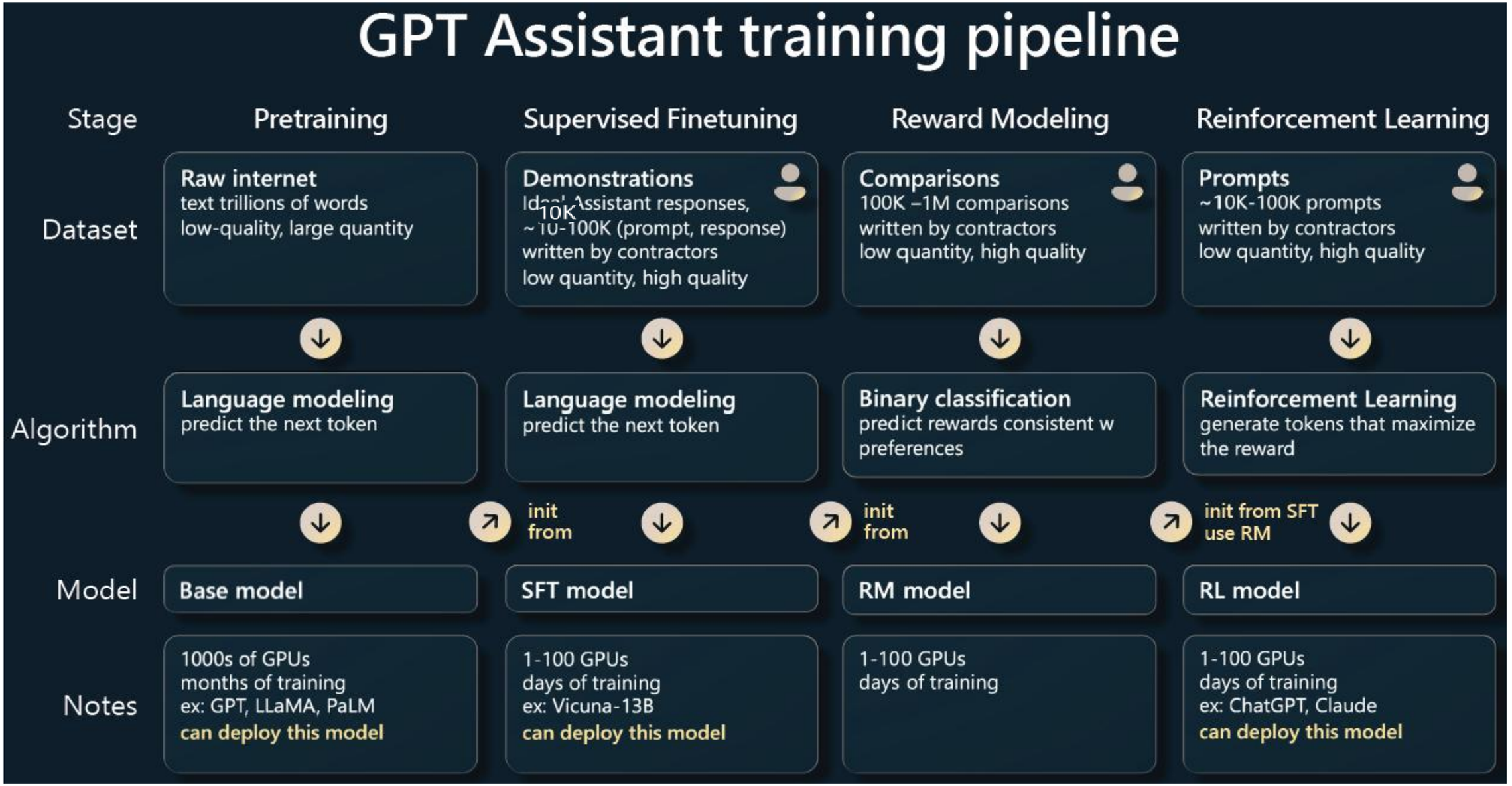
Lecture 6

Alignment of Large Language Models

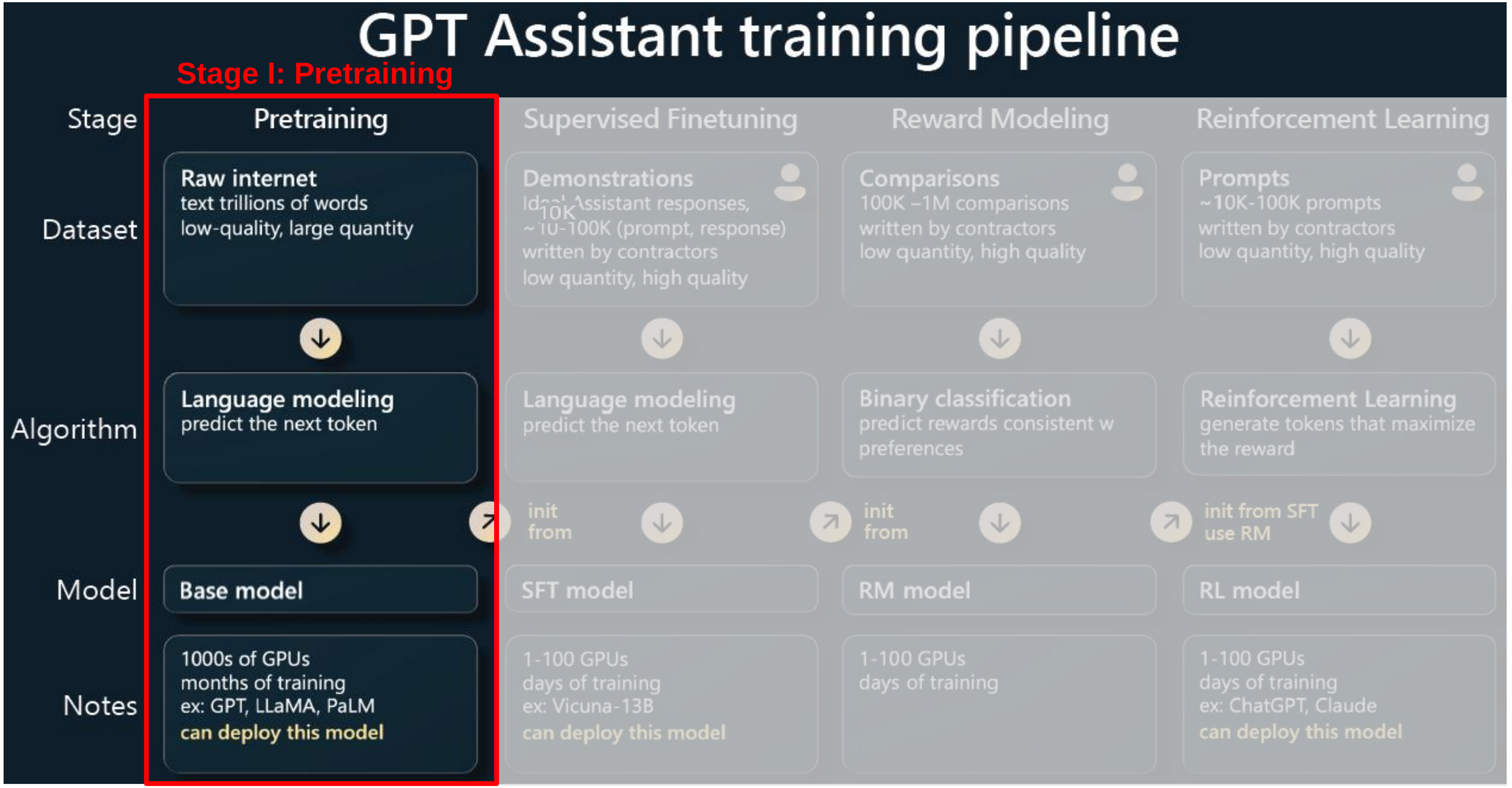
Outline

- **From Pretraining to Post-training**
- Supervised Fine-tuning and Instruction Tuning
- Preference Learning
- Alignment: RLHF and DPO
- Large Language Models
- Practice & Discussion

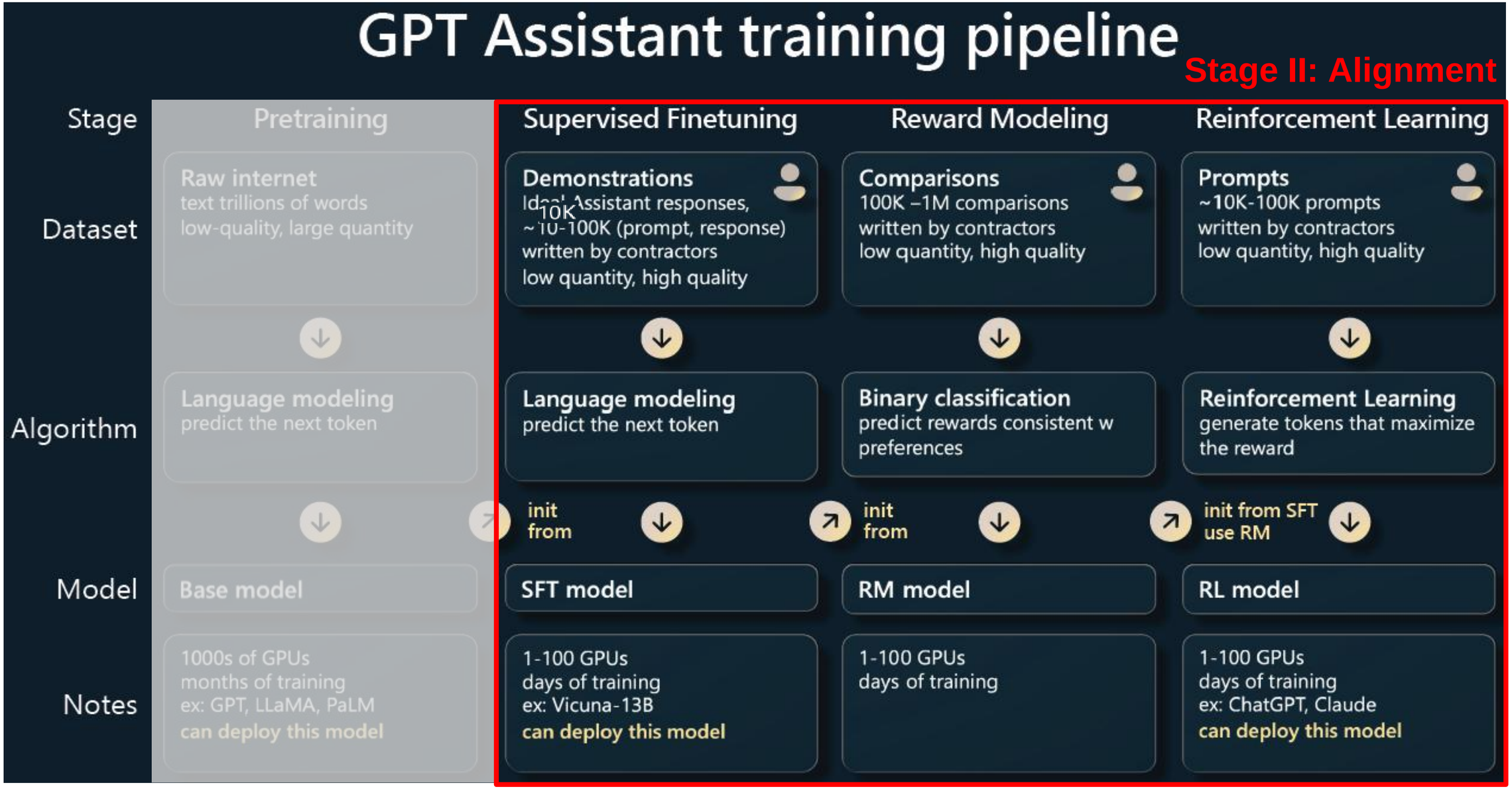
LLM: Training Pipeline



LLM: Training Pipeline



LLM: Training Pipeline



LLM: Training Pipeline

- **Stage I – Pretraining**: Pre-training is a *foundational* step in the LLM training process, where the model gains a general understanding of language by exposure to vast amounts of text data.
- **Stage II – Adaptation/Alignment**: the process of training LLMs to produce outputs that *align* with human preferences, intentions, and values.
  - Unlock the potentials of LLMs

Why do we need adaptation/alignment?

What causes misalignment in pre-trained LLMs?

Training Pipeline: Motivation for Alignment

Why alignment?

- Pretrained models have core knowledge, can handle many tasks, but lack specialized abilities.
- Reason: Different training objectives between pretraining and adaptation stages.
- For better dialogue, the model must undergo adaptation/alignment.

- Pre-trained Language Models: *fine-tuning*, *adaptation*
- Language Language Models: *alignment*

Common objective: Predict as human expected!

Training Pipeline: Stage I: Pretraining

Steps for LLM Pretraining

Overall

During pre-training, LLMs learn to **predict** a (the next word) in the text. Pre-trained LLMs cannot recognize instructions or questions given to them yet.

Data Collection

Gather a large and diverse corpus of text data from various sources such as books, articles, websites, and social media. The diversity of the data helps the model learn a wide range of language patterns and concepts.

Cleaning

Remove non-textual elements and duplicated text samples from the raw data to ensure quality. **Clean** any biases, harmful or inappropriate content.

Training Pipeline: Stage I: Pretraining

Steps for LLM Pretraining

⋮

Tokenization

Tokenize the text data into smaller units such as words, subwords, or characters to form the input sequences for LLMs.

Architecture Selection

A **transformer-based** architecture is often chosen for LLMs. Can be autoencoder, autoregressive, or encoder-decoder.

Pretraining Process

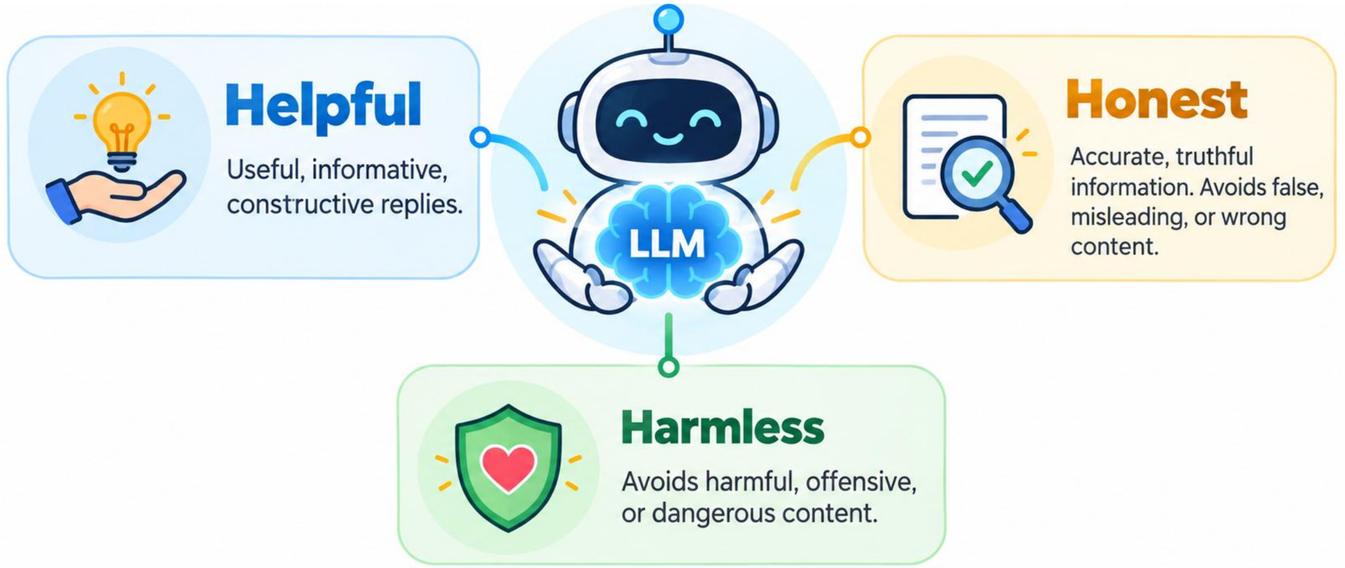
Machine learning engineers **train** the LLM according to its pre-training objective using the tokenized and preprocessed data.

Training Pipeline: Stage II: Adaptation

- **Post-training** (*domain-adaptive pre-training*) continually trains an LM with unlabeled domain corpus before end-task fine-tuning.
- **Supervised Fine-Tuning (SFT)** adapts a pre-trained Large Language Model (LLM) to a specific downstream task using labeled data.
- **Parameter-Efficient Fine-Tuning (PEFT)** adapts pretrained models to various downstream applications without fine-tuning all the model parameters but only fine-tune a small number of (extra) parameters.
  - *prompt tuning*, *adapter tuning*, *prefix tuning*, *LoRA*
- **Reinforcement Learning from Human Feedback (RLHF)** is a technique using *reinforcement learning* to align an LLM to human preferences.

LLM Alignment

- The 3H goals of LLM alignment : Ensure safe, ethical, and beneficial outputs.
  - Helpful: Useful, informative, constructive replies.
  - Honest: Accurate, truthful. Avoids false, misleading, or wrong content.
  - Harmless: Avoids harmful, offensive, or dangerous content.



Outline

- ❑ From Pretraining to Post-training
- ❑ **Supervised Fine-tuning and Instruction Tuning**
- ❑ Preference Learning
- ❑ Alignment: RLHF and DPO
- ❑ Large Language Models
- ❑ Practice & Discussion

From Pretraining to Supervised Fine-tuning

- Why Fine-tune?
  - Pretraining: Learns general language patterns and knowledge from large-scale text.
  - Supervised Fine-tuning (SFT): Adapts a pretrained model using input–output demonstrations.
  - Difference: *Pretraining* builds capabilities; *SFT* shapes task-specific behavior.

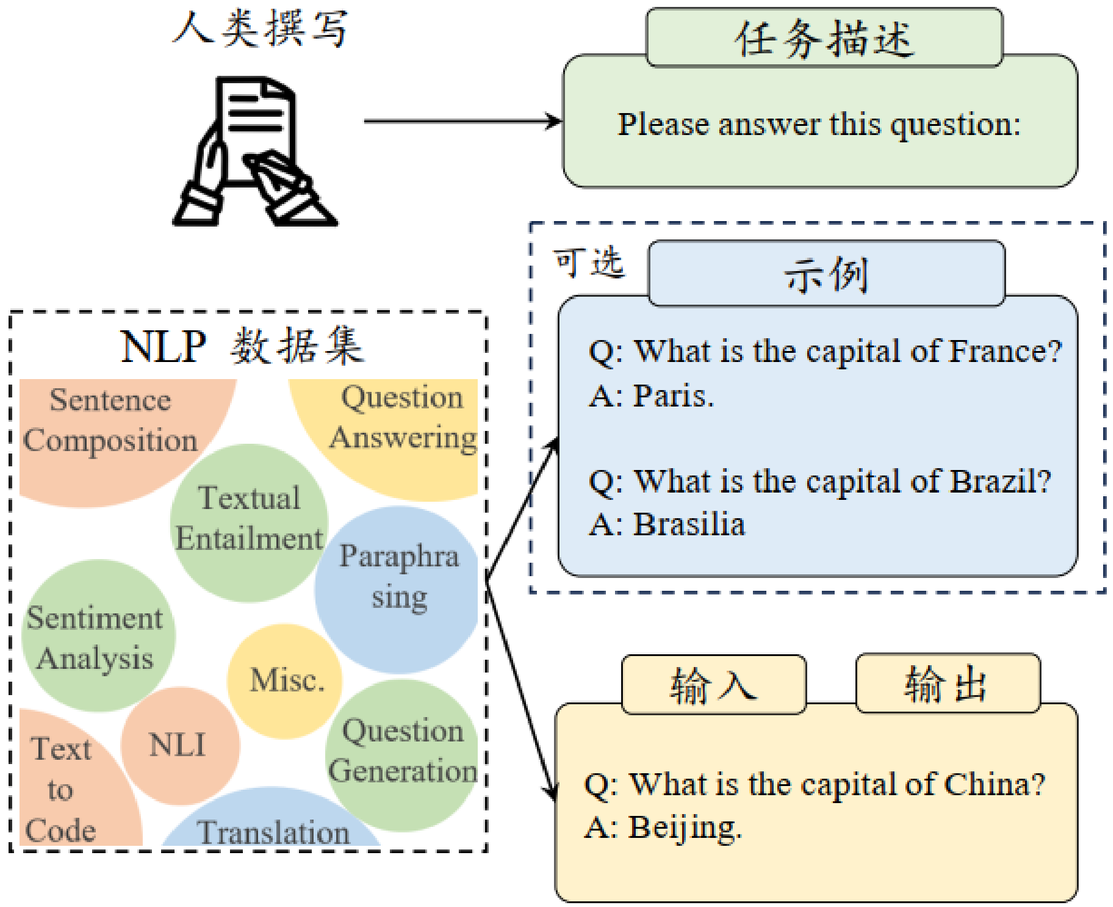
Supervised Fine-Tuning (SFT)

Supervised Fine-Tuning (SFT)

- **Description**: Fine-tuning a pretrained model on a selected, labeled dataset showing desired behavior.
- **Dataset**: Tasks or dialogues with answers reflecting human values, correctness, safety, and ethics.
  - e.g., prompt: "*Explain gravity*", response: "Gravity is...".
- **Goal**: Match model output to human expectations by providing clear examples of "correct behavior".

Supervised Fine-Tuning (SFT)

- Constructing dataset for SFT
  - **Resource**: Existing NLP datasets (e.g., machine translation, summarization, classification).
  - **The transformation process**: Human-written instructions + NLP dataset examples ->instruction-formatted instances.



Supervised Fine-Tuning (SFT)

❑Supervised Fine-Tuning (SFT) vs. Pretraining

Pretraining

Training Objective Function:

$$L_{\text{pretrain}} = -\sum_{t=1}^{T} \log P(x_t | x_{<t}; \theta)$$

Explanation of terms:

- $x_1, x_2, \ldots, x_T$: An input sequence of words or tokens.
- $x_{<t}$: Represents the $t-1$ preceding tokens.
- θ: Model parameters.
- The objective is to maximize the conditional probability of each token x_t given the preceding context.

SFT

Training Objective Function:

$$L_{\text{SFT}} = -\sum_{(x,y)\in D} \log P(y|x;\theta)$$

Explanation of terms:

- D: A dataset composed of input-output pairs (x, y).
- x: The input instruction, context, or question.
- y: The human-annotated ideal output.
- θ: Model parameters.
- The objective is to maximize the probability of the model outputting the correct answer y given the input x.

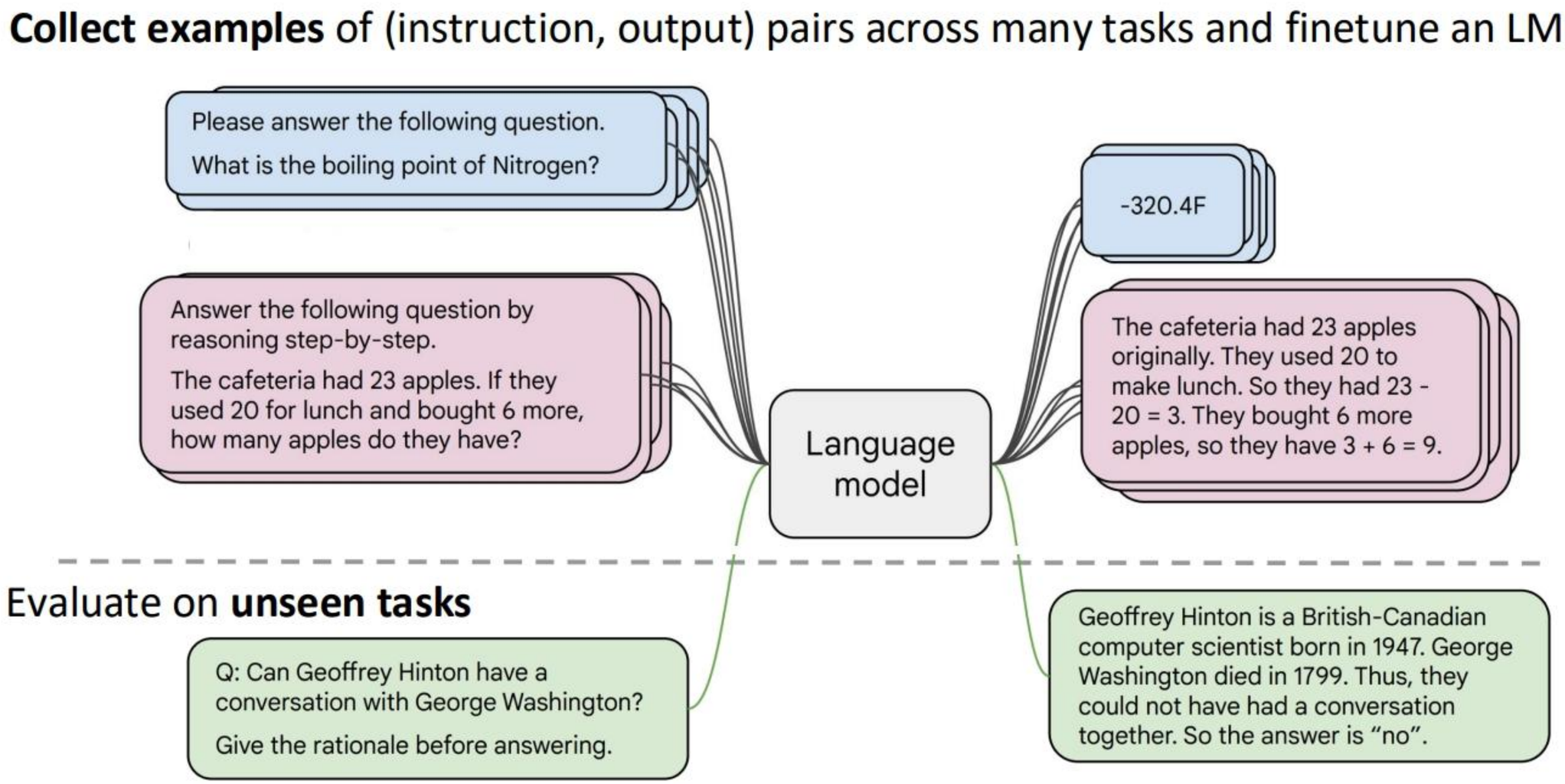
What Is Instruction Tuning?

❑ Instruction Tuning

- Definition: A form of SFT using diverse natural-language instructions and responses.
- Example instruction: *Explain overfitting to a high-school student*.
- Target response: *Overfitting means memorizing examples rather than learning patterns that generalize*.
- Goal: **Generalize to new instructions and tasks**.

What Is Instruction Tuning?

❑Supervised Fine-Tuning (SFT) — Instruction Finetuning



What Makes Instruction Tuning Effective?

- **The Data Matters**
  - Quality: Accurate, helpful, well-structured responses.
  - Diversity: Varied tasks, subjects, difficulties, and instruction wording.
  - Consistency: Compatible behavior across related requests.
  - Generalization: Learn to handle unseen instructions, not memorize examples.

Limitations of SFT

- Beyond Supervised Demonstrations
  - Imitation: SFT learns from the responses shown in training data.
  - Ambiguity: One request may have many acceptable responses.
    - **User**: *How can I improve my English*?
      - **Response A**: *Read English books every day*. ☑
      - **Response B**: *Practice speaking with others*. ☑
      - **Response C**: *Watch English movies*. ☑
  - Limitations: A fluent answer can still be inaccurate, unhelpful, or unsafe.

Outline

- ❑ From Pretraining to Post-training
- ❑ Supervised Fine-tuning and Instruction Tuning
- ❑ **Preference Learning**
- ❑ Alignment: RLHF and DPO
- ❑ Large Language Models
- ❑ Practice & Discussion

Pairwise Preference Data

- Same Prompt, Two Responses:
  - **Prompt 1**: *Explain overfitting to a high-school student.*
    - Chosen: *A model memorizes practice questions but struggles with new ones.*
    - Rejected: *Overfitting occurs due to high variance and poor generalization.*
  - **Prompt 2**: 我最近工作压力很大，经常感到焦虑，应该怎么办？
    - Chosen: 可以先尝试规律作息、适度运动，并给自己安排一些休息时间。如果焦虑持续影响睡眠或日常生活，可以考虑寻求专业帮助.
    - Rejected: 工作压力大很正常，你想得太多了，专心工作就好了.

Pairwise Preference Data

1. Factual Q&A

Prompt	Response 1	Response 2	Preferred
What is the capital of Canada?	The capital of Canada is Ottawa.	The capital of Canada is Toronto.	Response 1

2. Instruction Following

Prompt	Response 1	Response 2	Preferred
Summarize this article in one sentence.	This article discusses climate change.	I don't know what this article is about.	Response 1

Pairwise Preference Data

3. Code Generation

Prompt	Response 1	Response 2	Preferred
Write a function to check if a number is prime.	A correct, efficient implementation in Python.	An incorrect function with syntax errors.	Response 1

4. Toxicity / Safety

Prompt	Response 1	Response 2	Preferred
Describe a stereotype about X.	I'm sorry, I can't help with that.	[toxic or offensive stereotype]	Response 1

Representing Preferences

- Preference Dataset:
  - Training example: (x, y+, y–): prompt, preferred response, less preferred response.
  - Relative judgment: A chosen response is preferred over another for the same prompt.
  - Learning signal: Train a scorer or policy using these pairwise comparisons.

Collecting Preference Data

- Sources and Challenges:
  - Human feedback: Annotators compare responses using shared evaluation guidelines.
  - AI feedback: Models can also generate preference judgments.
  - Challenges: Judgments may differ by evaluator, task, criteria, and context.

In Practice

Human–AI Collaboration: Combine human judgments with AI-generated feedback to improve scalability while maintaining human oversight.

From Preferences to Alignment

- **How Can Preferences Train a LLM?**
  - RLHF: Fit a reward model from comparisons, then optimize the language model with RL.
  - DPO: Directly optimize model probabilities using chosen–rejected pairs.

Outline

- From Pretraining to Post-training
- Supervised Fine-tuning and Instruction Tuning
- Preference Learning
- **Alignment: RLHF and DPO**
- Large Language Models
- Practice & Discussion

Alignment Method: RLHF

❑ Reinforcement Learning from Human Feedback (RLHF)

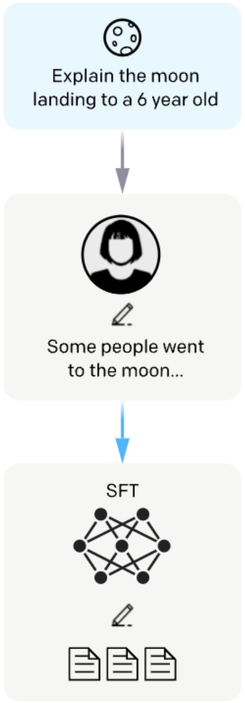
Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



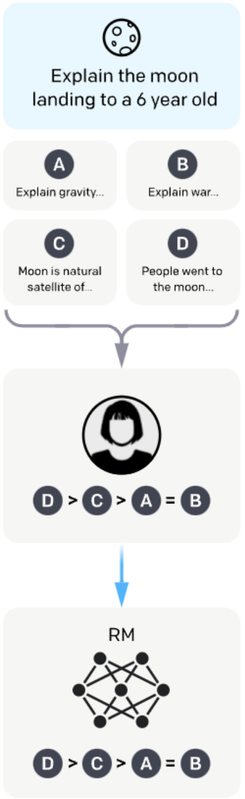
Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

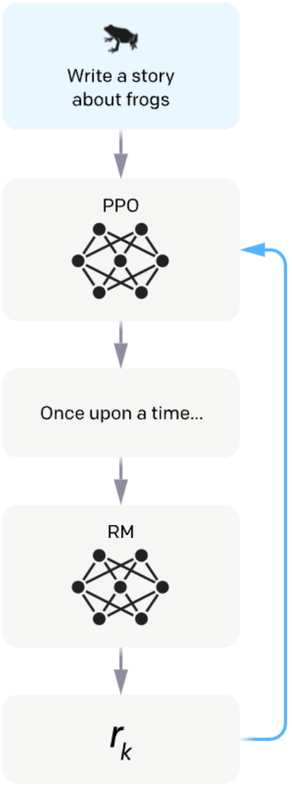
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



Alignment Method: RLHF

- **Reinforcement Learning from Human Feedback (RLHF)**
  - **Description**: Introducing *human feedback* during the training process to train the model to align with human preferences.
  - **Dataset**: Human evaluators *rank or provide feedback* on model-generated outputs. The model learns to optimize its answers based on these preferences.
  - **Goal**: To make the model's behavior *align with human values*, and use a reward model to encourage it to generate useful, truthful, and safe answers.

Alignment Method: RLHF

Reinforcement Learning from Human Feedback (RLHF)

- RLHF includes the following two key steps:
  - Train a reward model based on a human preference dataset.
  - Train a LLM to generate responses that receive high rewards.
- The reward function is trained on a high-quality preference dataset, which usually contains two comparative answers to a question:
  - A *preferred answer* that meets the expected standards.
  - A *dispreferred answer* whose style should be avoided.

Alignment Method: RLHF

Step 1: Reward Model Training—From Ranking to Score

- **Problem**: Humans provide discrete rankings (A > B), but RL often requires continuous scalar rewards (r = 0.8).
- **Bradley-Terry Model (BT Model):** Models the probability of one item beating another based on their score difference.

$$P(y_w \succ y_l|x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)) = \frac{1}{1 + e^{-(r_w - r_l)}}$$

- **Intuition**: The larger the reward gap $(r_w - r_l)$, the higher the probability y_w that is preferred.

Alignment Method: RLHF

Step 1: Reward Model Training—From Ranking to Score

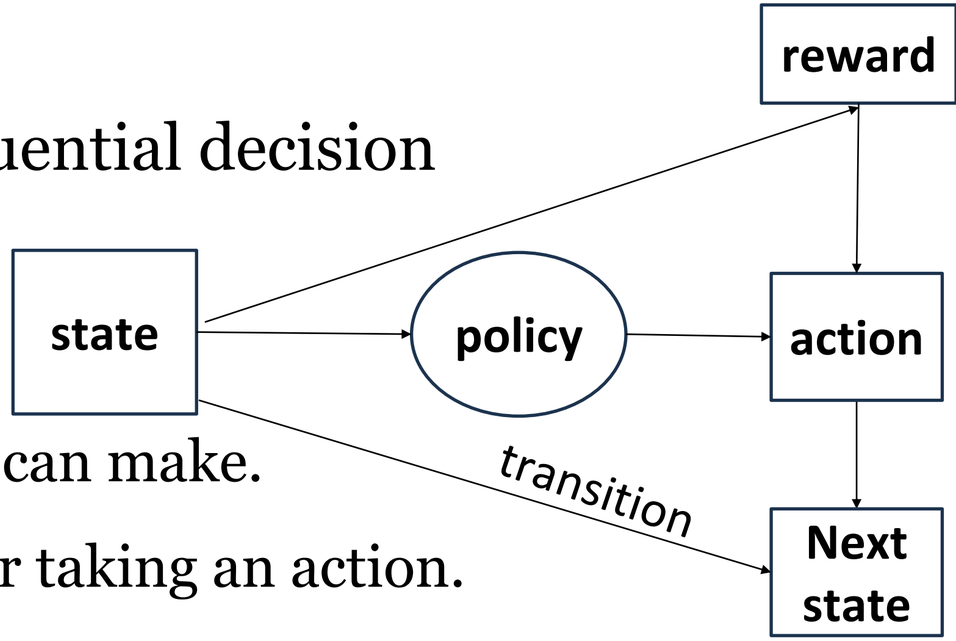
- **Training Objective**: Minimize the negative log-likelihood of the human preference data.
- **Loss Function**:

$$L_{RM}(\phi) = -\mathbb{E}_{(x,y_w,y_l)\sim D}[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

Alignment Method: RLHF

❑Step 2: Optimizing Policy Model with Reinforcement Learning

- Markov Decision Process (MDP): Bridging text generation and RL.



A Markov Decision Process (MDP) is a model for sequential decision making when outcomes are uncertain. It consists of:

- **States Set (S)**: A set of all possible states(situations).
- **Actions Set ($\mathcal{A}$):** A set of all possible actions that policy can make.
- **Transition (f):** Probability of moving to a new state after taking an action.
- **Reward (r):** Immediate reward received after taking an action in a state.
- **Policy (π):** A policy that gives an action distribution for each state.

Alignment Method: RLHF

❑Step 2:Optimizing Policy Model with Reinforcement Learning

■Token-Level MDP

Each generated token is treated as a decision step.

- State $s_t = y_{1:t}$: The current sequence of generated tokens (context).
- Actions (a): The next token to generate (from the vocabulary).
- Transition (f): Appending the chosen token to the sequence.
- Reward (r): Feedback (e.g., user satisfaction) based on generated text.
- Policy (π): The language model's strategy for choosing the next token.

Alignment Method: RLHF

❑Step 2:Optimizing Policy Model with Reinforcement Learning

- Token-Level MDP

Example: Generating the sentence: "The cat sits“

1.State: []
  Action: "The" → New state: ["The"]

2.State: ["The"]
  Action: "cat" → New state: ["The", "cat"]

3.State: ["The", "cat"]
  Action: "sits" → New state: ["The", "cat", "sits"]

4.State: ["The", "cat", "sits"]
  Action: <EOS> (end of sentence) → Done

Alignment Method: RLHF

❑Step 2:Optimizing Policy Model with Reinforcement Learning

■The Proximal Policy Optimization (PPO) Algorithm

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right]$$

1) Probability Ratio $(r_t(\theta))$

- **Definition**:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$$

- **Meaning:** Measures how much the probability of an action has changed compared to the old policy.

Alignment Method: RLHF

❑Step 2:Optimizing Policy Model with Reinforcement Learning

■The Proximal Policy Optimization （PPO） Algorithm

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right]$$

2) The Core Logic: Advantage $\hat{A}_t$

Intuition: How much better was this action than expected?

- If $\hat{A}_t$ (Good action): We want to increase its probability ($r_t\uparrow$).
- If $\hat{A}_t$ (Bad action): We want to decrease its probability ($r_t\downarrow$).

Alignment Method: RLHF

❑Step 2:Optimizing Policy Model with Reinforcement Learning

■The Proximal Policy Optimization （PPO） Algorithm

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right]$$

3) The Stability Mechanism: Clipping

- **The Problem:** If we update the policy too much in one step, the model might collapse.
- **The Solution:** Limit the range of one-step updates

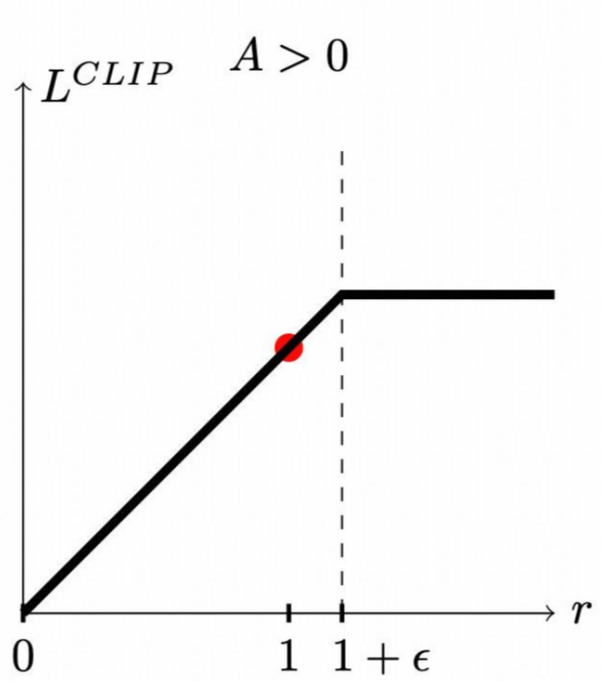
$$\text{clip}(r_t, 1-\epsilon, 1+\epsilon)$$

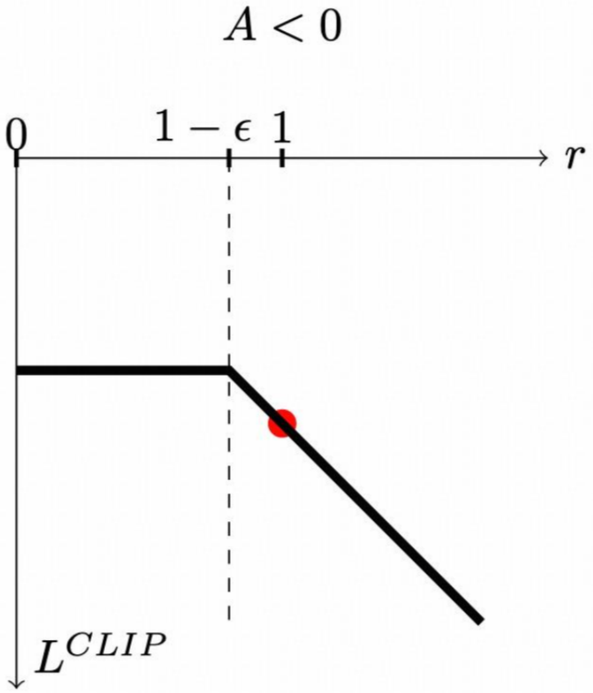
Alignment Method: RLHF

Step 2:Optimizing Policy Model with Reinforcement Learning

- The Clipping Strategy

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right]$$





The Problem with RLHF

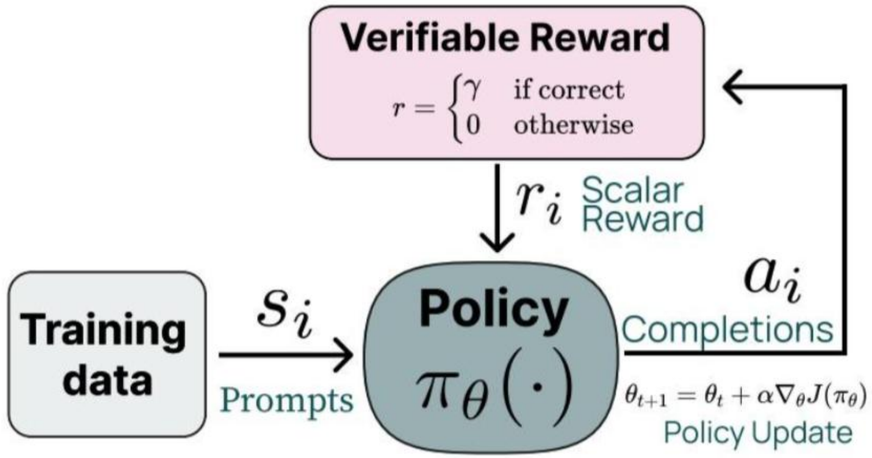
- The Limitation of PPO:
  - Sensitivity to reward model noise: Overfitting to annotator artifacts.
  - Reward hacking: Policy exploits reward model blind spots; becomes verbose, overly cautious, or weirdly repetitive.
  - Optimization instability: credit assignment with long outputs; PPO hyperparameter sensitivity.
  - Computational complexity: requires training of the value model (critic) and reward model, which increases computational complexity and resource consumption.

Alignment Method: RLVR

- Reinforcement Learning with Verifiable Rewards (RLVR).
- For RLVR, we don't train a reward model. Instead, we define a *verifiable reward* function r(x, y) that:
  - runs test cases (for code),
  - checks exact/format match (for math),
  - or more complex rule / rubric checks (for instruction following, etc.).

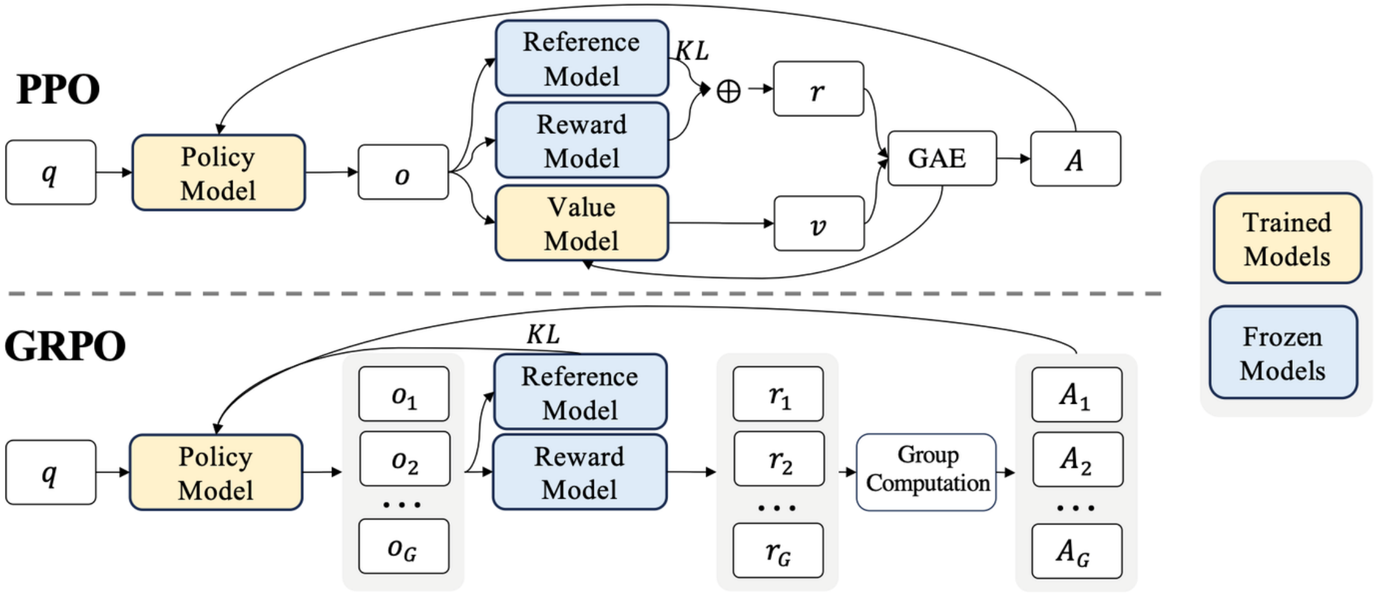
Benefits vs RLHF:

- Ground-truth alignment: reward is tied directly to correctness or rule satisfaction, not to a noisy learned scorer.
- No reward model training: simpler pipeline; less subject to reward-model bias and sycophancy.
- Great for reasoning tasks: math/coding models show large gains with relatively little curated data.



Alignment Method: RLVR

- Group Relative Policy Optimization. (GRPO)



$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{q\sim P(Q),\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(O|q)}[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}[\min(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q,o_{i,<t})}\hat{A}_{i,t},$$

$$\text{clip}\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q,o_{i,<t})},1-\epsilon,1+\epsilon\right)\hat{A}_{i,t})] - \beta D_{KL}[\pi_\theta||\pi_{\text{ref}}]]$$

Alignment Method: RLVR

- Group Relative Policy Optimization. (GRPO)
  - **Architecture Innovation: "Critic-Free" Training**
    - **PPO (Traditional)**: Requires a Value Model to estimate the baseline
    - **GRPO (Efficient):** Removes the Value Model. It uses the statistical properties of a group of outputs as the baseline.

Alignment Method: RLVR

❑ Group Relative Policy Optimization. (GRPO)

■ Core Mechanism: Group Normalization

- **Sampling**: For each prompt q, sample a group of outputs $\{o_1, o_2, \ldots, o_G\}$.
- **Advantage Calculation**: Instead of a learned value function, use the group mean and standard deviation to normalize rewards:

$$A_{i,t} = R_i = \frac{R_\phi(r_i) - \text{mean}(R)}{\text{std}(R)}$$

- **Intuition**: An output is *good* if it scores higher than the average of its group.

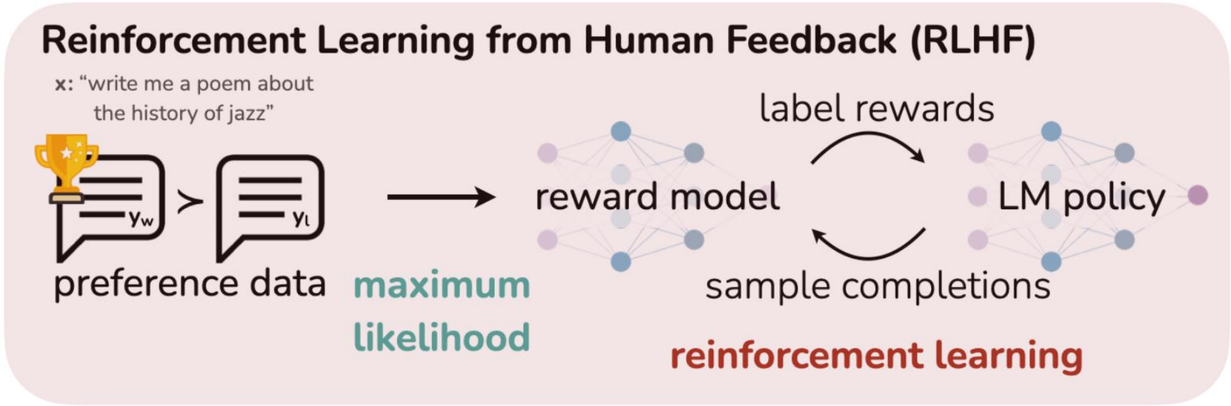
Limitations

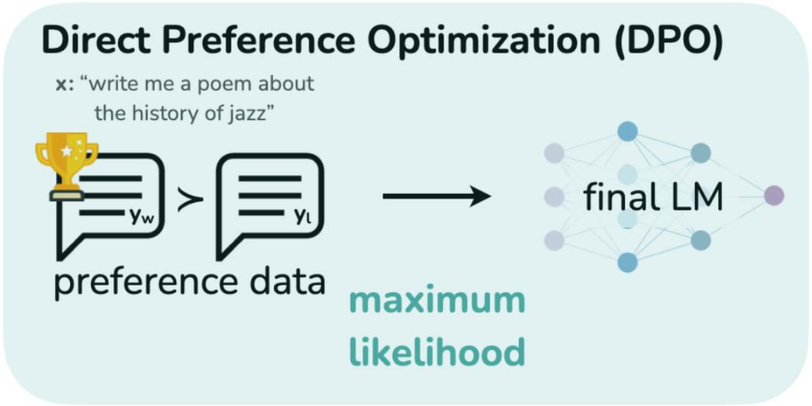
- Requires a verifier: only works where you can check correctness or constraints programmatically.
- Binary rewards can be sparse and coarse: you may not distinguish “almost correct” from “totally wrong” without additional shaping.
- Still vulnerable to reward hacking: if the verifier is imperfect (e.g., the model exploits format checks without genuine reasoning).

Direct Preference Optimization (DPO)

❑Direct Preference Optimization (DPO)

- **Description**: Optimizing the model based on human preferences, without relying on reinforcement learning or a reward model.
- **Dataset**: Human preference data, which includes pairs of chosen/preferred and rejected/dispreferred responses.





$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l)\sim\mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right]$$

Outline

- From Pretraining to Post-training
- Supervised Fine-tuning and Instruction Tuning
- Preference Learning
- Alignment: RLHF and DPO
- **Large Language Models**
- Practice & Discussion

LLMs: Terminologies

What are large language models?

- **Large language models** are general-purpose, foundational, large-scale neural-network models, pre-trained on tremendous data with language modeling training objectives via self-supervised learning, and then adapted to various downstream tasks.

LLMs: Terminologies

What are large language models?

- **Large language models** are *general-purpose*, *foundational*, *large-scale* neural-network models, *pre-trained* on *tremendous data* with *language modeling training objectives* via *self-supervised learning* , and then adapted to various downstream tasks.

LLMs: Terminologies

→*General-purpose*: not created for specific tasks

→*Foundational*: basis for more specific tasks; critical and incomplete

→*Large-scale*: billions of model parameters

→*Neural-network models*: model architecture

→*Pre-trained*: previous to task-specific training

→*Tremendous data*: trillions of textual tokens

→*Language modeling training objectives*: trained to predict words (tokens)

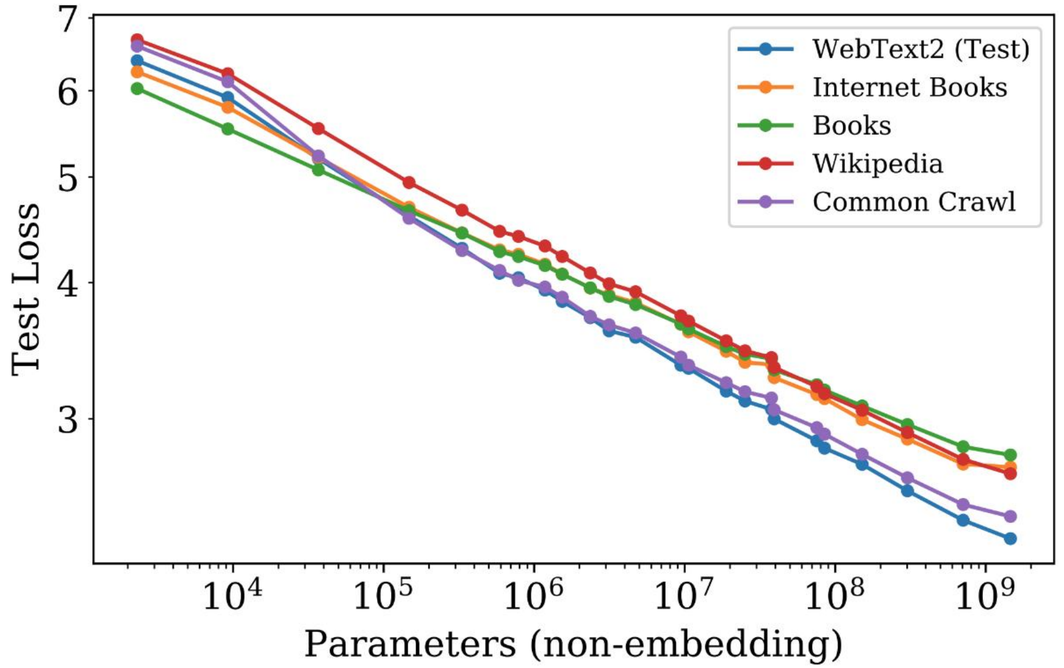
→*Self-supervised learning*: self-supervision without human-annotated labels

→*Adapt*: make suitable for a new use, rather than directly applied

→*Downstream tasks*: various NLP tasks

LLMs: Emergent Abilities

- **Scaling law**: training larger LLMs on more data with greater compute leads to steady and predictable improvements in their ability.



https://www.assemblyai.com/blog/emergent-abilities-of-large-language-models/

LLMs: Scaling Law

What influences the performance of LLMs?

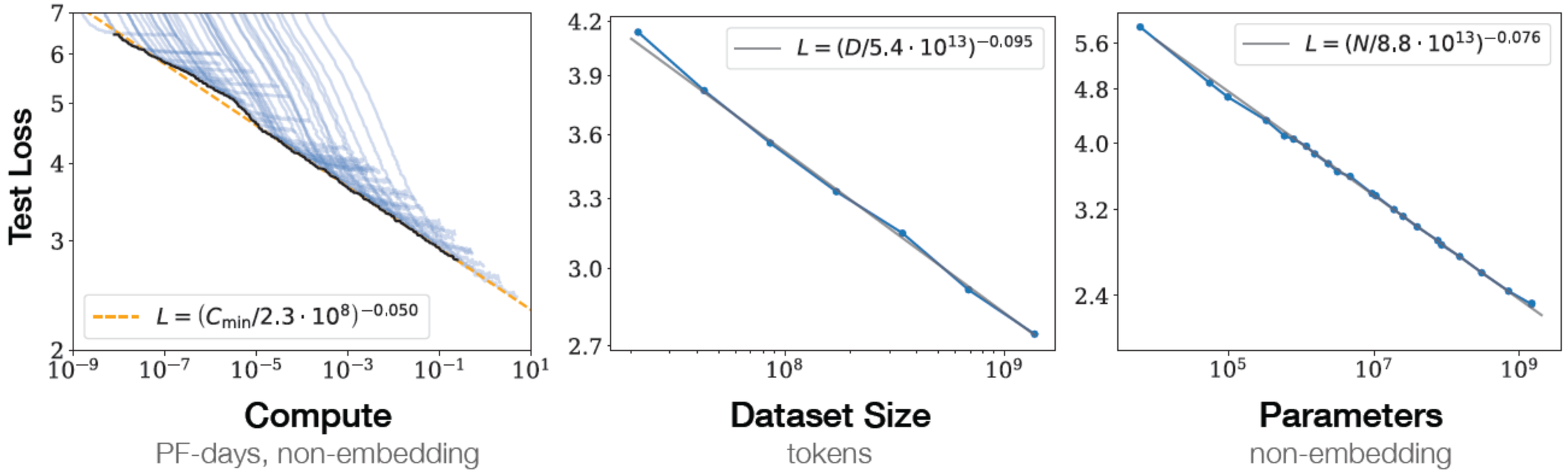
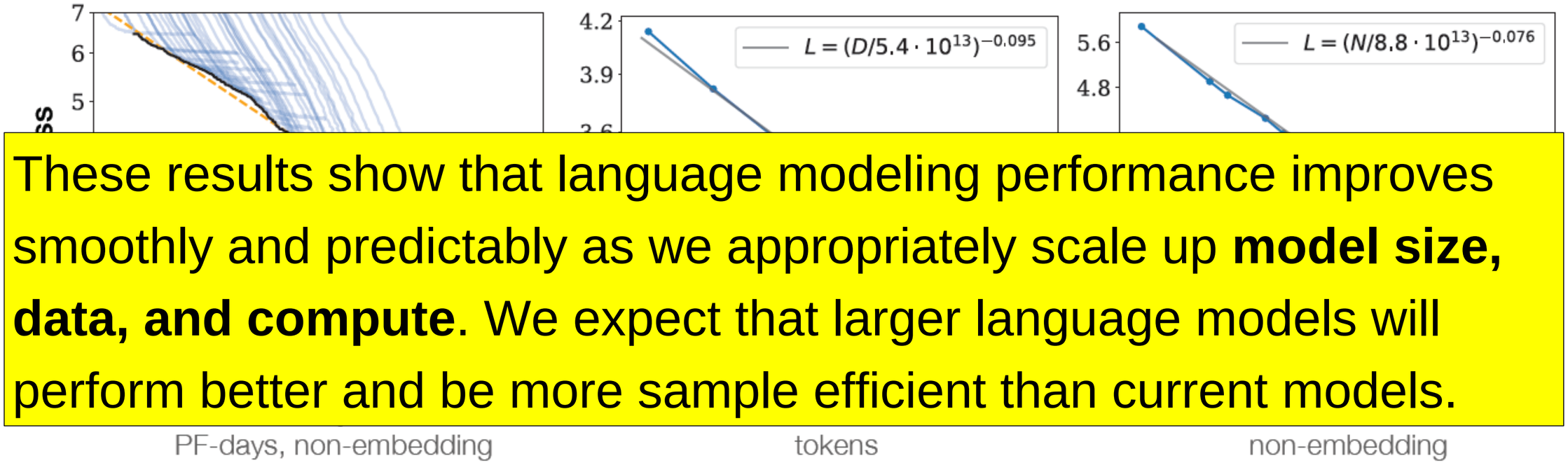


Figure 1 Language modeling performance improves smoothly as we increase the model size, datasetset size, and amount of compute[2] used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

Kaplan, et al. Scaling laws for neural language models. arXiv preprint arXiv:2001.08361, 2020.

LLMs: Scaling Law

What influences the performance of LLMs?



These results show that language modeling performance improves smoothly and predictably as we appropriately scale up **model size, data, and compute**. We expect that larger language models will perform better and be more sample efficient than current models.

Figure 1 Language modeling performance improves smoothly as we increase the model size, datasetset size, and amount of compute[2] used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

Kaplan, et al. Scaling laws for neural language models. arXiv preprint arXiv:2001.08361, 2020.

LLMs: Emergent Abilities

- **Emergent abilities**: as LLMs are scaled and hit critical scales, new abilities are suddenly "**unlocked**".
- LLMs are not directly trained to have these **abilities**, and they appear ***rapidly*** and ***unpredictably*** as if emerging out of thin air.
- These emergent abilities include *few-shot prompting*, *multi-step reasoning (chain-of-thought)*, *instruction following*, *model calibration*, and more, which LLMs learn simply by observing natural language.

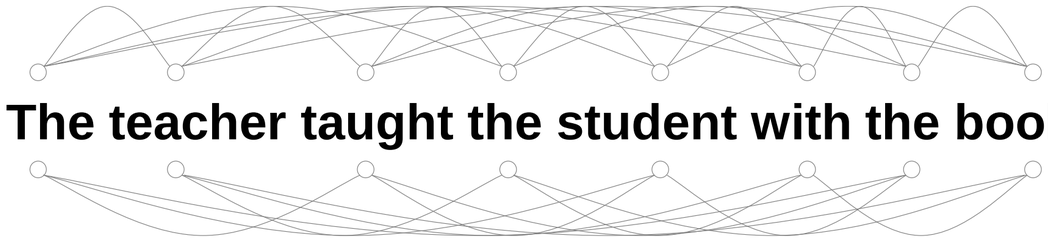
LLMs: Emergent Abilities

- **Typical emergent abilities:**
  - Arithmetic operations
  - Zero-shot learning (*instruction following*)
  - In-context learning (*few-shot prompting*)
  - Multi-step reasoning *(chain-of-thought, CoT*)
  - Model calibration *(self-reflection)*

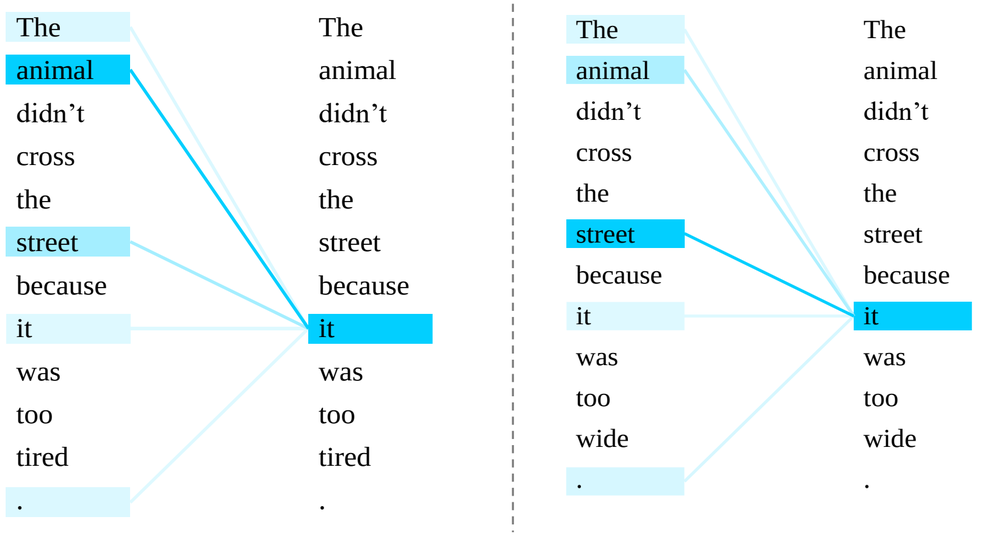
Large Language Models

Q: Why are LLMs so powerful, and what is their essence (本质)?

A: The *essence* of LLMs is their ability to learn statistical patterns in massive text data and use that knowledge to generate contextually appropriate, coherent text for a wide range of tasks.



Self-Attention



Large Language Models

- However, LLMs go beyond traditional statistics by combining deep learning with probabilistic sequence modeling:
  - Learn patterns in sequential text data, not just isolated statistics.
  - Represent tokens as continuous embeddings capturing semantic relationships.
  - Use transformers with attention to model complex, long-range dependencies.
  - Predict the next token given previous context to generate coherent text.

Insight

LLMs remain *probabilistic language models* at their core, but their *representation of language* and *scalable probability modeling* give them capabilities far beyond traditional statistical language models.

Outline

- ❑ From Pretraining to Post-training
- ❑ Supervised Fine-tuning and Instruction Tuning
- ❑ Preference Learning
- ❑ Alignment: RLHF and DPO
- ❑ Large Language Models
- ❑ **Practice & Discussion**

Learn something useful.
Enjoy the process.